

Teoría de Recursividad - Ejercicios Conceptuales (01-04)

Este documento resume los fundamentos de la recursividad según la guía de la Escuela de Computación.

1. Concepto de Recursividad y Niveles

Recursividad: Es una técnica de programación donde una función se define en términos de sí misma. Para que sea válida, debe tener:

- **Caso Base:** Una condición de parada que devuelve un valor sin hacer más llamadas recursivas.
- **Paso Recursivo:** Una llamada a la misma función con un argumento que acerca el problema al caso base.

Niveles de Recursividad: Se refiere a la profundidad de la recursión o cuántas veces se ha llamado la función a sí misma antes de alcanzar el caso base. Cada nivel representa una instancia pendiente en la pila de ejecución.

Ejemplo: En el factorial de 3 (3!), el nivel 0 es `fact(3)`, el nivel 1 es `fact(2)`, y el nivel 2 es `fact(1)`.

2. Ventajas y Desventajas

Ventajas	Desventajas
Simplicidad: Soluciones más naturales para problemas matemáticos o estructuras jerárquicas (árboles).	Eficiencia: Cada llamada consume memoria en la pila (Stack).
Legibilidad: El código suele ser más corto y fácil de entender si el problema es recursivo por naturaleza.	Riesgo de desbordamiento: Si la recursión es muy profunda, puede ocurrir un <i>Stack Overflow</i> .

3. Pilas de Recursividad y Ambientes

Cuando una función se llama recursivamente, el sistema utiliza una **Pila de Control (System Stack)** para gestionar la ejecución:

- **Ambiente Recursivo:** Cada llamada crea un "ambiente" privado con sus propias copias de variables locales y parámetros.
 - **Pila de Recursividad:** El estado de la función actual se "empuja" (push) a la pila mientras se espera el resultado de la llamada hija. Cuando la hija termina, el estado se "saca" (pop) para continuar donde se dejó.
-

4. ¿Recursividad o Iteración?

¿Es mejor la iteración? Sí, en muchos casos prácticos.

- **Rendimiento:** La iteración (bucles `for/while`) no tiene el costo de gestión de la pila ni el tiempo de creación de ambientes.
- **Cuándo elegir iteración:** Para problemas simples (como recorrer un arreglo o sumar números) o cuando la memoria es crítica.
- **Cuándo elegir recursividad:** Cuando el problema es intrínsecamente recursivo (como recorrer un sistema de archivos o el algoritmo Quicksort) donde la versión iterativa sería extremadamente compleja de implementar.